# BCS703 – MODULE 3

Public-Key Distribution • X.509 Certificates • Public-Key Infrastructure (PKI)

| # | Topic |
|---|---|
| 1 | Distribution of Public Keys (4 schemes) |
| 2 | Public-Key Certificates |
| 3 | X.509 Certificates (format, chain, revocation, version 3) |
| 4 | Public-Key Infrastructure (PKIX model) |
| 5 | Secret-key distribution using public-key encryption (supporting topic) |
| 6 | Textbook review questions (Ch.14) with answers |
| 7 | Last-minute revision sheet |

**How to use:** every topic has (1) a quick definition, (2) the steps/points in the order the textbook gives them, (3) the textbook figure, and (4) an **Exam keywords** box. Write the bold words in your answer to score marks. Content and figures are taken only from your uploaded textbook pages.

# 1. DISTRIBUTION OF PUBLIC KEYS (Section 14.3)

**Problem:** the public key is "public", but the receiver must be sure that the key really belongs to the claimed person. Otherwise a forger can pose as user A. **Four general schemes** are used for public-key distribution:

1. **Public announcement**
2. **Publicly available directory**
3. **Public-key authority**
4. **Public-key certificates**

## 1.1 Public Announcement of Public Keys

- Any participant can send his/her public key to any other participant or **broadcast** it to the community (e.g., a broadly accepted algorithm such as **RSA**).
- Example: many **PGP** users append their public key to messages sent to public forums such as **USENET newsgroups** and Internet mailing lists.
- **Advantage:** convenient.
- **Major weakness: anyone can forge** such an announcement. Some user can pretend to be user A and send/broadcast a public key. Until A discovers the forgery and alerts others, the forger can **read all encrypted messages intended for A** and can use the forged keys for authentication.




Figure 14.10 Uncontrolled Public-Key Distribution

*Fig 14.10 Uncontrolled Public-Key Distribution*

## 1.2 Publicly Available Directory

A greater degree of security by maintaining a **publicly available dynamic directory** of public keys. Maintenance and distribution of the directory is the responsibility of a **trusted entity or organization**. Elements of the scheme:

1. The authority maintains a directory with a **{name, public key}** entry for each participant.
2. Each participant **registers** a public key with the directory authority. Registration is **in person** or by some form of **secure authenticated communication**.
3. A participant may **replace** the existing key with a new one at any time – because the key has already been used for a large amount of data, or because the private key has been **compromised**.
4. Participants can also access the directory **electronically**; for this, **secure, authenticated communication** from the authority to the participant is mandatory.

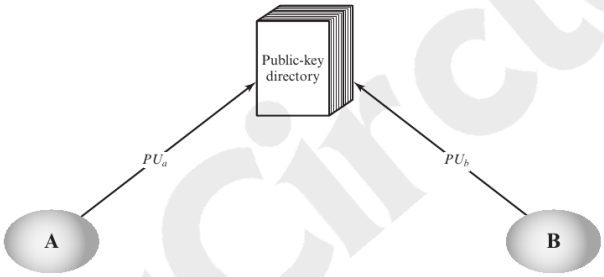



Figure 14.11 Public-Key Publication

*Fig 14.11 Public-Key Publication*

**Vulnerabilities:** more secure than public announcement, but if an adversary **obtains or computes the private key of the directory authority**, the adversary can pass out **counterfeit public keys**, impersonate any participant and eavesdrop. The adversary can also **tamper with the records** kept by the authority.

## 1.3 Public-Key Authority

Stronger security by providing **tighter control** over distribution of keys from the directory. A central authority maintains a dynamic directory of public keys of all participants. Each participant **reliably knows the public key of the authority** (only the authority knows the private key $PR_{auth}$).

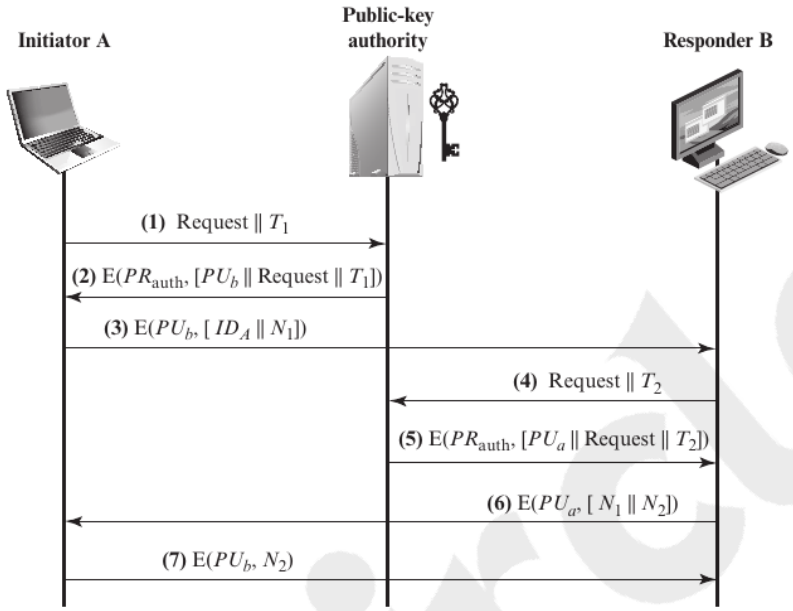



Figure 14.12 Public-Key Distribution Scenario

*Fig 14.12 Public-Key Distribution Scenario (7 messages)*

1. A sends a **timestamped** message to the authority requesting the current public key of B → **Request || T1**.
2. The authority responds with a message **encrypted with the authority's private key $PR_{auth}$**: E($PR_{auth}$, [$PU_b$ || Request || T1]). A decrypts with the authority's public key, so A is sure the message came from the authority. It contains: (i) **B's public key $PU_b$**; (ii) the **original request** – lets A match the response and verify the request was not altered; (iii) the **original timestamp** – shows this is not an old message with a stale key.
3. A stores $PU_b$ and uses it to encrypt a message to B containing **$ID_A$** and a **nonce N1**: E($PU_b$, [$ID_A$ || N1]).
4. **4, 5.** B retrieves A's public key from the authority in the same manner (Request || T2; E($PR_{auth}$, [$PU_a$ || Request || T2])).
5. B sends A: E($PU_a$, [N1 || N2]). Because only B could decrypt message (3), presence of N1 **assures A that the correspondent is B**.
6. A returns **N2** encrypted with $PU_b$: E($PU_b$, N2) – **assures B** that its correspondent is A.

- Total **seven messages**. The first five are needed only infrequently because A and B can **save (cache)** each other's public key; periodically request fresh copies to ensure **currency**.
- **Drawbacks:** the authority can be a **bottleneck** (every user must appeal to it for every other user it wants to contact); the directory of names and public keys is **vulnerable to tampering**.

## 1.4 Public-Key Certificates (Kohnfelder)

Certificates let participants **exchange keys without contacting a public-key authority**, in a way as reliable as if the keys were obtained directly from an authority. A **certificate = public key + identifier of key owner + whole block signed by a trusted third party** (a **Certificate Authority, CA**, e.g., a government agency or financial institution trusted by the user community).

- A user presents his/her public key to the CA in a **secure manner** and obtains a certificate, then **publishes** it. Anyone needing this key obtains the certificate and **verifies it by the attached trusted signature**. A participant can also send its certificate directly to another.

### Requirements of the certificate scheme

1. Any participant can **read** a certificate to determine the **name and public key** of the certificate's owner.
2. Any participant can **verify** that the certificate originated from the CA and is **not counterfeit**.
3. Only the **CA can create and update** certificates.
4. Any participant can verify the **time validity** of the certificate (added by **Denning**).

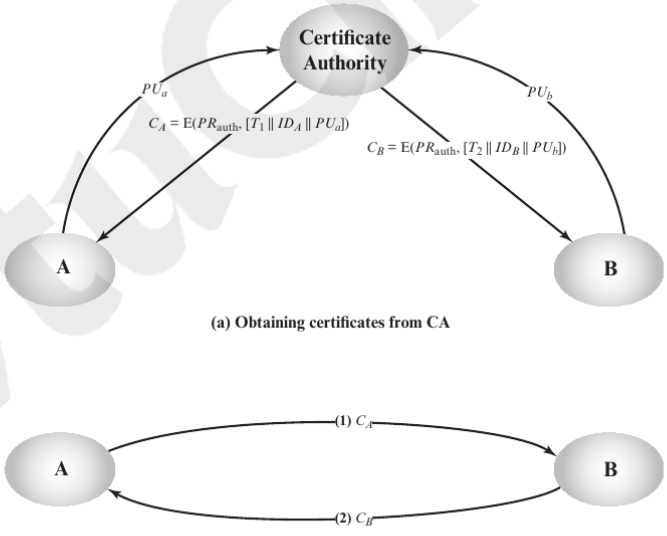



Figure 14.13 Exchange of Public-Key Certificates

*Fig 14.13 Exchange of Public-Key Certificates*

**Certificate for A:** $C_A = E(PR_{auth}, [ T || ID_A || PU_a ])$ where $PR_{auth}$ = CA's private key, T = timestamp.

**Verification by recipient:** $D(PU_{auth}, C_A) = D(PU_{auth}, E(PR_{auth}, [T || ID_A || PU_a])) = (T || ID_A || PU_a)$

- Readable **only with the CA's public key** → proves it came from the CA. $ID_A$ and $PU_a$ give the name and public key of the holder; the **timestamp T validates currency**.
- **Why timestamp?** Suppose A's private key is learned by an adversary. A generates a new key pair and applies for a new certificate. Meanwhile the adversary **replays the old certificate** to B; if B encrypts with the old (compromised) key, the adversary reads the messages. The timestamp acts like a **credit-card expiration date**: a sufficiently old certificate is assumed **expired**.
- One scheme is **universally accepted** for formatting public-key certificates: the **X.509 standard** (used in IPsec, TLS, S/MIME).

**Exam keywords – Public-key distribution**
public announcement • public directory • {name, public key} • trusted entity • public-key authority • PR_auth • timestamp • nonce • 7 messages • caching • bottleneck • certificate • CA • C_A = E(PR_auth,[T||ID_A||PU_a]) • forgery • man-in-the-middle • time validity

# 2. X.509 CERTIFICATES (Section 14.4)

- **ITU-T recommendation X.509** is part of the **X.500** series that defines a **directory service** (a server or distributed set of servers that maintains a database of information about users – mapping from user name to network address and other attributes).
- X.509 defines a **framework for authentication services** by the X.500 directory. The directory serves as a **repository of public-key certificates**; each certificate contains the **public key of a user** and is **signed with the private key of a trusted CA**. It also defines alternative authentication protocols based on public-key certificates.
- Important because the certificate format is used in **S/MIME, IP Security and SSL/TLS**.
- Issued first in **1988**; revised in **1993**; currently **version 7 (2012)**. Based on **public-key cryptography and digital signatures**; does **not dictate** a specific signature algorithm or hash function.

## 2.1 How a certificate is created and used (Fig 14.14)

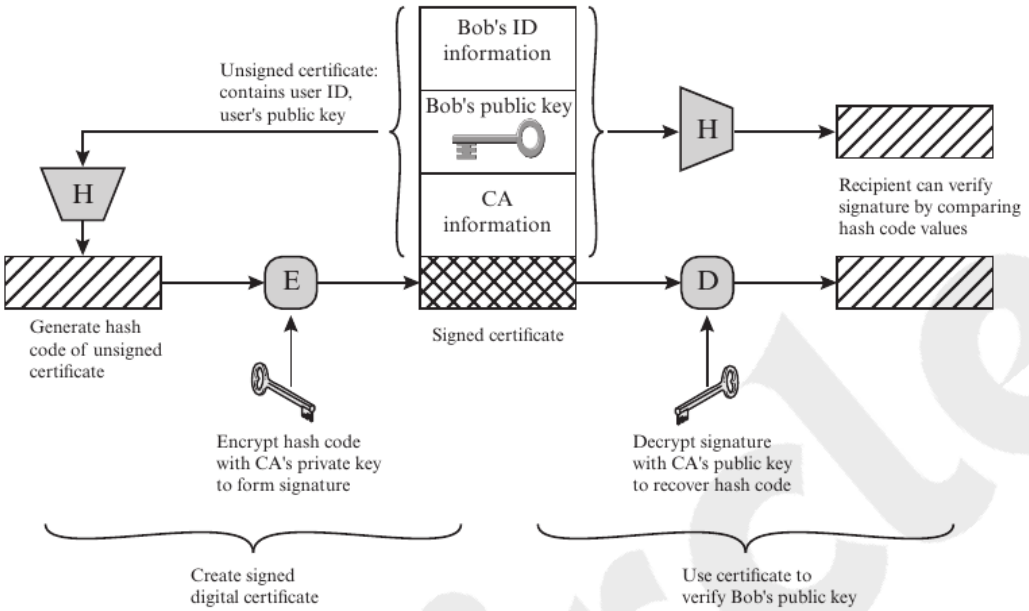



Figure 14.14 X.509 Public-Key Certificate Use

*Fig 14.14 X.509 Public-Key Certificate Use*

1. The unsigned certificate contains **Bob's ID information, Bob's public key and CA information**.
2. A **hash code** of the unsigned certificate is generated.
3. The hash code is **encrypted with the CA's private key** to form the **signature** → signed certificate.
4. Recipient: **decrypts the signature with CA's public key** to recover the hash code, computes the hash of the certificate itself and **compares the two hash values**. If equal, Bob's public key is verified.

If **NIST DSA** or **ECDSA** is used, the hash value is **not encrypted** but is input to a digital-signature generation algorithm.

## 2.2 Certificate format (Fig 14.15a)

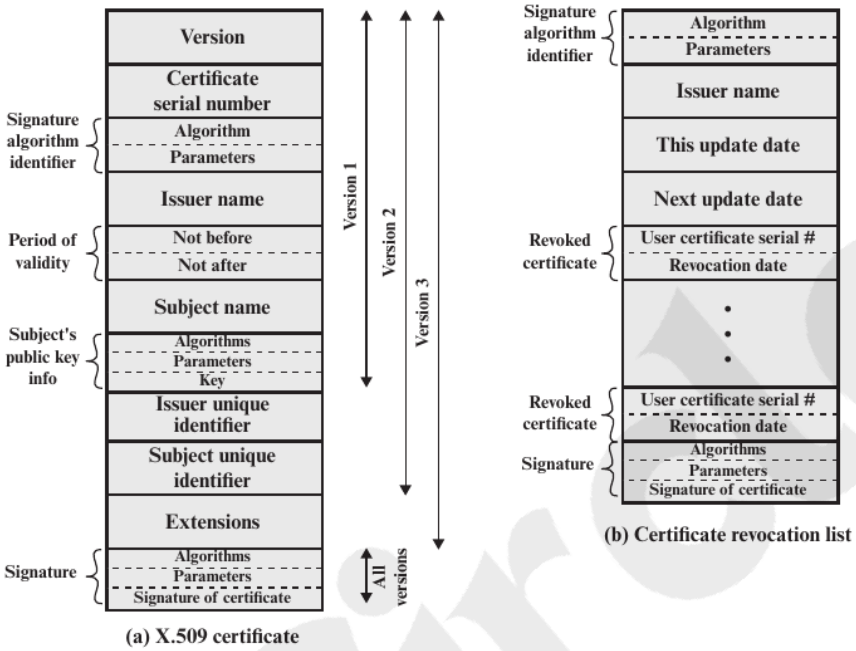



Figure 14.15 X.509 Formats

*Fig 14.15 X.509 Formats – (a) certificate (b) certificate revocation list*

| Field | Meaning (textbook) |
|---|---|
| **Version** | Differentiates successive versions; default is **version 1**. If *issuer/subject unique identifier* present → **version 2**. If one or more **extensions** present → **version 3**. |
| **Serial number** | Integer value **unique within the issuing CA**, unambiguously associated with this certificate. |

| Field | Meaning (textbook) |
|---|---|
| **Signature algorithm identifier** | Algorithm (with parameters) used to sign the certificate. Repeated in the signature field, so little utility. |
| **Issuer name** | X.500 name of the CA that created and signed the certificate. |
| **Period of validity** | Two dates: **Not before** and **Not after** – first and last day the certificate is valid. |
| **Subject name** | Name of the user to whom the certificate refers (holder of the corresponding private key). |
| **Subject's public-key info** | Public key of the subject + identifier of the algorithm for which the key is used + parameters. |
| **Issuer unique identifier** | Optional bit string to uniquely identify the issuing CA if the X.500 name has been reused (added in v2; rarely used). |
| **Subject unique identifier** | Optional bit string to uniquely identify the subject if the X.500 name has been reused (v2; rarely used). |
| **Extensions** | One or more extension fields (added in **v3**). |
| **Signature** | Covers all other fields; contains the **digital signature** applied to the other fields + signature algorithm identifier. |

**X.509 notation:** CA<<A>> = CA { V, SN, AI, CA, UCA, A, UA, Ap, $T^A$ }

| Symbol | Meaning |
|---|---|
| Y<<X>> | certificate of user X issued by certification authority Y |
| Y{I} | signing of I by Y = I with an **encrypted hash code appended** |
| V | version |
| SN | serial number |
| AI | algorithm identifier used to sign |
| CA / UCA | name of CA / optional unique identifier of CA |
| A / UA | name of user A / optional unique identifier of A |
| Ap | public key of user A |
| $T^A$ | period of validity |

## 2.3 Obtaining a user's certificate

- Characteristics of CA-generated certificates: (i) any user with the CA's public key can **verify** the user public key that was certified; (ii) **no party other than the CA can modify** the certificate without detection.
- Because certificates are **unforgeable**, they can be placed in a directory without the directory needing special protection.
- If all users subscribe to the **same CA** → common trust. With a large community, many **CAs** are practical, each securely providing its public key to some fraction of users.

### Certificate chain (A has certificate from X1, B from X2)

1. A gets from the directory the **certificate of X2 signed by X1**; since A knows X1's public key securely, A obtains X2's public key and verifies it with X1's signature.
2. A gets the **certificate of B signed by X2**; with trusted X2 public key A verifies and obtains B's public key.

Chain: **X1<<X2>> X2<<B>>**; reverse chain for B: X2<<X1>> X1<<A>>. N-element chain: X1<<X2>> X2<<X3>> ... $X_N$<<B>> (each pair of CAs must have created certificates for each other). X.509 suggests arranging CAs in a **hierarchy** for easy navigation.

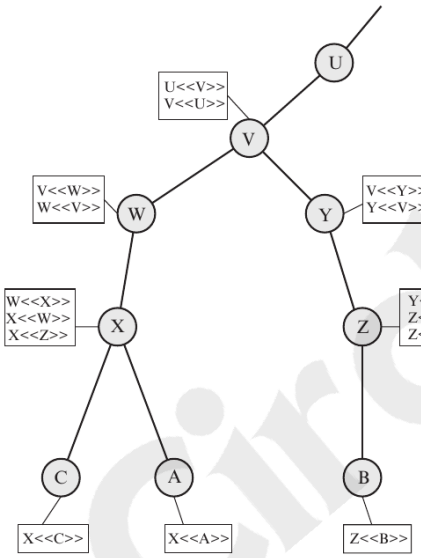



Figure 14.16 X.509 Hierarchy: A Hypothetical Example

*Fig 14.16 X.509 Hierarchy: A Hypothetical Example*

- Each CA directory entry has two types of certificates: **Forward certificates** – certificates of X generated by other CAs; **Reverse certificates** – certificates generated by X that are the certificates of other CAs.
- Path from A to B in the figure: **X<<W>> W<<V>> V<<Y>> Y<<Z>> Z<<B>>**. A unwraps the path in sequence to get a trusted copy of B's public key. For B to get A's key: Z<<Y>> Y<<V>> V<<W>> W<<X>> X<<A>>.

## 2.4 Revocation of certificates

Each certificate has a period of validity (like a credit card). A new one is issued just before expiry. A certificate may be **revoked before it expires** for these reasons:

1. The user's **private key is assumed compromised**.
2. The user is **no longer certified by this CA** (subject's name changed, certificate superseded, or not issued in conformance with CA policies).
3. The **CA's certificate is assumed compromised**.

- Each CA maintains a list of all **revoked but not expired** certificates (issued to users and other CAs) = **Certificate Revocation List (CRL)**, posted on the directory.
- CRL is **signed by the issuer** and includes (Fig 14.15b): **issuer's name, date list was created (this update), date next CRL is scheduled (next update), and an entry for each revoked certificate** = **serial number + revocation date**. Serial numbers are unique within a CA, so serial number is enough.
- On receiving a certificate, the user must check whether it is revoked; to avoid delay/cost of directory search, the user keeps a **local cache** of certificates and CRLs.

## 2.5 X.509 Version 3

Version 2 does not convey all information needed. **[FORD95]** lists requirements not met: (1) subject field inadequate to convey identity of key owner; (2) subject field inadequate for apps that use email address/URL; (3) need to indicate **security policy** information (e.g., for IPSec); (4) need to **limit damage from a faulty or malicious CA** by constraints; (5) need to identify **different keys used by same owner at different times** (key lifecycle management).

Solution: version 3 adds optional **extensions**. Each extension = **extension identifier + criticality indicator + extension value**. If the criticality indicator is **TRUE** and the implementation does not recognise the extension, it **must treat the certificate as invalid**. Three categories:

| Category | Extensions (textbook) |
|---|---|
| **Key and policy information** (info about subject and issuer keys + certificate policy = named set of rules for applicability to a community/class of application) | **Authority key identifier** (handles CA key-pair updating); **Subject key identifier**; **Key usage** (digital signature, nonrepudiation, key encryption, data encryption, key agreement, CA signature verification on certificates/CRLs); **Private-key usage period**; **Certificate policies**; **Policy mappings** (only in CA certificates issued by other CAs) |

| Category | Extensions (textbook) |
|---|---|
| **Certificate subject and issuer attributes** (alternative names/extra info to increase confidence about the subject) | **Subject alternative name** (email, EDI, IPSec name forms); **Issuer alternative name**; **Subject directory attributes** (X.500 attribute values) |
| **Certification path constraints** (constraints in certificates issued for CAs by other CAs) | **Basic constraints** (may the subject act as a CA? path-length constraint); **Name constraints** (name space for subject names in later certificates); **Policy constraints** (require explicit policy identification / inhibit policy mapping) |

**Exam keywords – X.509**
ITU-T X.509 • X.500 directory • version 1/2/3 • serial number • issuer name • period of validity • subject name • unique identifier • extensions • signature • CA<<A>> • forward certificate • reverse certificate • certificate chain • CRL • revoked but not expired • criticality indicator • key usage • basic constraints

# 3. PUBLIC-KEY INFRASTRUCTURE – PKI (Section 14.5)

**Definition (RFC 4949 Internet Security Glossary):** PKI is the **set of hardware, software, people, policies and procedures needed to create, manage, store, distribute and revoke digital certificates** based on **asymmetric cryptography**.

**Principal objective:** to enable **secure, convenient and efficient acquisition of public keys**. The **IETF PKIX (Public Key Infrastructure X.509) working group** set up a formal, generic model based on X.509 suitable for a certificate-based architecture on the Internet.

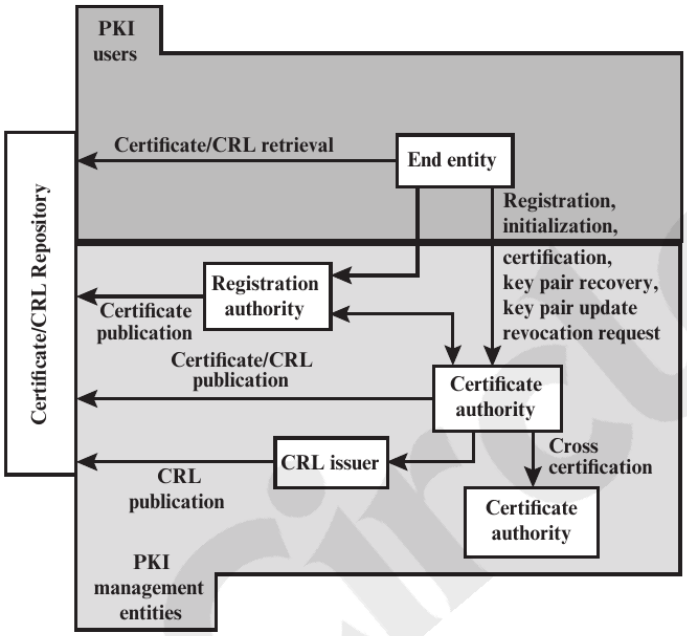



Figure 14.17 PKIX Architectural Model

*Fig 14.17 PKIX Architectural Model*

## 3.1 Key elements of the PKIX model

| Element | Role |
|---|---|
| **End entity** | Generic term for end users, devices (servers, routers) or any entity that can be identified in the subject field of a public-key certificate. Typically **consume and/or support PKI-related services**. |
| **Certification authority (CA)** | **Issuer of certificates and (usually) CRLs**. May support administrative functions, often delegated to RAs. |
| **Registration authority (RA)** | **Optional** component that can assume administrative functions from the CA; often associated with the **end-entity registration process**. |
| **CRL issuer** | **Optional** component a CA can delegate to **publish CRLs**. |
| **Repository** | Any method for **storing certificates and CRLs** so end entities can retrieve them. |

### 3.2 PKIX management functions

| Function | Meaning |
|---|---|
| **Registration** | User first makes itself known to a CA (directly or through an RA) **before** the CA issues a certificate; begins enrolling in the PKI. Involves offline/online procedure for **mutual authentication**; end entity typically gets one or more **shared secret keys**. |
| **Initialization** | Before a client can operate securely, install key material with the right relationship to keys elsewhere – e.g., client is securely initialized with the **public key of trusted CA(s)** to validate certificate paths. |
| **Certification** | CA **issues a certificate** for a user's public key, returns it to the user's client and/or **posts it in a repository**. |
| **Key pair recovery** | Recover the decryption keys when normal access to keying material is no longer possible (forgotten passwords/PINs, corrupted disk, damaged tokens). Entities restore their key pair from an **authorized key backup facility** (typically the CA). |
| **Key pair update** | All key pairs must be updated regularly (replaced with new pair and new certificates) – when **certificate lifetime expires** and after **revocation**. |
| **Revocation request** | Authorized person advises CA of an abnormal situation requiring revocation: **private-key compromise, change in affiliation, name change**. |
| **Cross certification** | Two CAs exchange information to create a **cross-certificate** = a certificate issued by one CA to another CA containing a CA signature key used for issuing certificates. |

### 3.3 PKIX management protocols

- **RFC 2510 – Certificate Management Protocols (CMP):** each management function is explicitly identified by specific protocol exchanges; designed to be **flexible**, accommodating many technical, operational and business models.
- **RFC 2797 – Certificate Management Messages over CMS (CMC):** CMS = RFC 2630 **Cryptographic Message Syntax**. Built on earlier work to **leverage existing implementations**; all PKIX functions are supported but do not all map into specific protocol exchanges.

**Exam keywords – PKI**
RFC 4949 • hardware, software, people, policies, procedures • create, manage, store, distribute, revoke • PKIX • end entity • CA • RA (optional) • CRL issuer • repository • registration • initialization • certification • key pair recovery • key pair update • revocation request • cross certification • CMP RFC 2510 • CMC RFC 2797

## 4. SUPPORTING TOPIC – SECRET-KEY DISTRIBUTION USING PUBLIC-KEY ENCRYPTION (Section 14.2)

Public-key systems are inefficient for large data blocks, so one of their most important uses is to **encrypt secret keys for distribution**.

### 4.1 Simple secret-key distribution (Merkle)

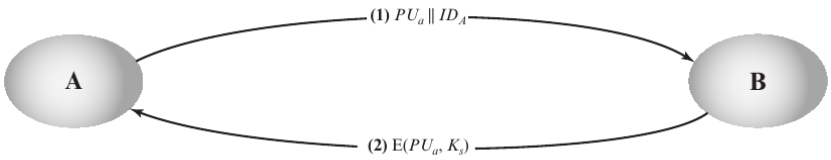



Figure 14.7 Simple Use of Public-Key Encryption to Establish a Session Key

*Fig 14.7 Simple Use of Public-Key Encryption to Establish a Session Key*

1. A generates a public/private pair {$PU_a$, $PR_a$} and sends B a message with **$PU_a$ || $ID_A$**.
2. B generates secret key **$K_s$** and sends it to A encrypted with A's public key: $E(PU_a, K_s)$.
3. A computes $D(PR_a, E(PU_a, K_s))$ to recover $K_s$. Only A can decrypt, so only A and B know $K_s$.
4. A discards $PU_a$ and $PR_a$; B discards $PU_a$.

- Attractive: **no keys exist before or after** the communication, so risk of compromise is minimal; secure against **eavesdropping**.
- **Insecure against man-in-the-middle attack** (active adversary).

## 4.2 Man-in-the-middle attack on this protocol (Fig 14.8)

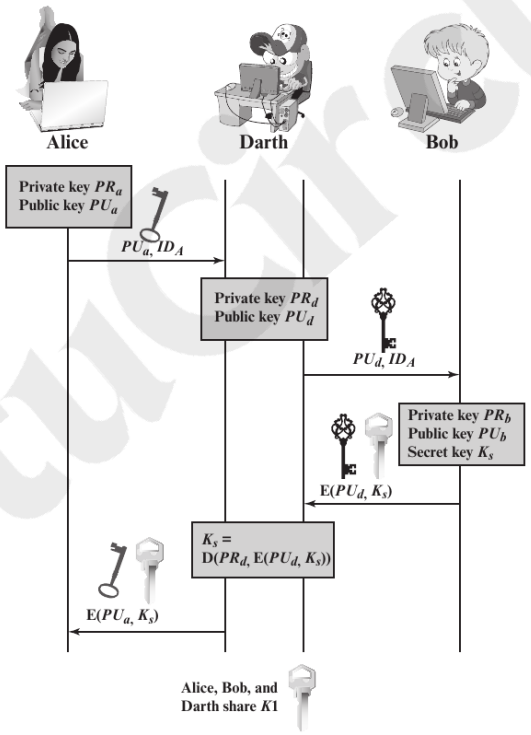



Figure 14.8 Another Man-in-the-Middle Attack

*Fig 14.8 Another Man-in-the-Middle Attack*

1. A sends {$PU_a$, $ID_A$} intended for B.
2. Darth (D) intercepts, creates own pair {$PU_d$, $PR_d$}, and sends **$PU_d$ || $ID_A$** to B.
3. B generates $K_s$ and transmits $E(PU_d, K_s)$.
4. D intercepts and learns $K_s = D(PR_d, E(PU_d, K_s))$.
5. D transmits $E(PU_a, K_s)$ to A.

Result: A and B both know $K_s$ and are unaware D also knows it; D only eavesdrops and decrypts all messages. So the simple protocol is only useful where the **only threat is eavesdropping**.

## 4.3 Secret-key distribution with confidentiality and authentication (Fig 14.9, Needham)

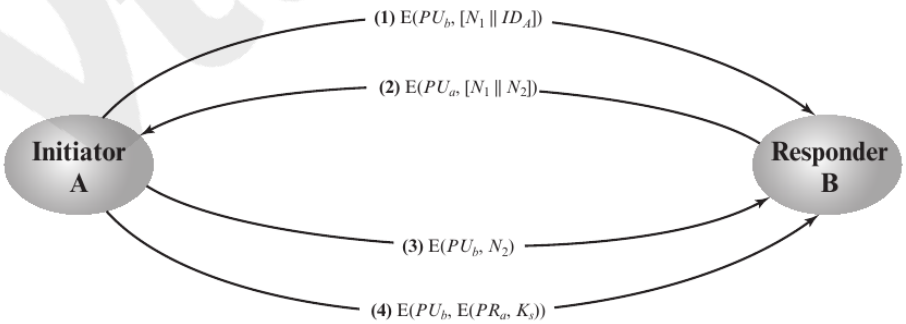



Figure 14.9 Public-Key Distribution of Secret Keys

*Fig 14.9 Public-Key Distribution of Secret Keys*

Assumes A and B have already exchanged public keys. Gives protection against **both active and passive attacks**.

1. A → B: **$E(PU_b, [N1 || ID_A])$** – identifier of A and nonce N1 identifying the transaction.
2. B → A: **$E(PU_a, [N1 || N2])$** – only B could decrypt (1), so N1 in (2) assures A that the correspondent is B.
3. A → B: **$E(PU_b, N2)$** – assures B that its correspondent is A.
4. A selects secret key $K_s$ and sends **$M = E(PU_b, E(PR_a, K_s))$**. Encryption with B's public key ensures only B can read it; with A's private key ensures only A could have sent it.

5. B computes **$D(PU_a, D(PR_b, M))$** to recover the key.

### 4.4 Hybrid scheme (IBM mainframes)

- Retains a **KDC** that shares a secret **master key** with each user and distributes **session keys encrypted with the master key**; a **public-key scheme distributes the master keys** (three-level approach).
- **Performance:** session keys change frequently; public-key encryption for each would degrade performance, so public-key encryption is used only **occasionally to update the master key**.
- **Backward compatibility:** easily overlaid on an existing KDC scheme with minimal disruption.

## 5. TEXTBOOK REVIEW QUESTIONS (Ch.14) – MODEL ANSWERS

| Question | Answer (from textbook content) |
|---|---|
| 14.2 Major issue in end-to-end key distribution? How does key hierarchy help? | In a distributed system each host may talk to many others, so each device needs many keys supplied dynamically; with N hosts **N(N−1)/2 keys** are needed (e.g., 1000 nodes → ~half a million keys; 10,000 applications → up to 50 million). **Key hierarchy:** a KDC distributes **session keys** encrypted by a **master key** shared with each entity; only **N master keys** are needed, and they can be delivered by non-cryptographic means (e.g., physical delivery). |
| 14.3 What is a nonce? | A **unique identifier for a transaction** (timestamp, counter or random number). Minimum requirement: it **differs with each request**; it should be hard to guess, so a **random number** is a good choice. It protects against **replay** and masquerade. |
| 14.4 What is a KDC? | A **key distribution center** responsible for distributing session keys to pairs of users (hosts, processes, applications) as needed; each user shares a **unique master key** with the KDC. It must be **trusted and protected from subversion**. |
| 14.5 Two uses of public-key cryptography related to key distribution? | (1) **Distribution of public keys** (announcement, directory, authority, certificates). (2) **Use of public-key encryption to distribute secret keys** (Section 14.2). |
| 14.1 Why are MITM attacks ineffective on the protocol of Fig 14.3? | In Fig 14.3 the session key $K_s$ is sent from the KDC encrypted under the **master keys $K_a$ / $K_b$, which are pre-shared and not sent**, and **nonces** (N1, N2) plus the authentication steps 4–5 bind the messages. No public keys are exchanged unauthenticated, so an interceptor like D (Fig 14.8) has nothing to substitute. |

## LAST-MINUTE REVISION SHEET – MODULE 3

| Topic | One-line recall |
|---|---|
| Public announcement | Broadcast key – **easily forged**. |
| Public directory | Trusted authority keeps {name, key}; weak if authority private key/records are compromised. |
| Public-key authority | **7 messages**, timestamps + nonces, PR_auth signs replies; **bottleneck**; caching. |
| Certificate | C_A = E(PR_auth,[T\|\|ID_A\|\|PU_a]); no online authority needed. |
| X.509 | v1/v2/v3; fields: version, serial no., sig alg, issuer, validity, subject, subject public key info, unique IDs, extensions, signature. |
| Chain | X1<<X2>> X2<<B>>; forward and reverse certificates; hierarchy. |
| Revocation | CRL signed by issuer; reasons: key compromise, no longer certified, CA compromised. |
| PKI | People+policies+procedures+hardware+software to create/manage/store/distribute/revoke certificates; EE, CA, RA, CRL issuer, repository. |
| Mgmt functions | Registration, initialization, certification, key-pair recovery, key-pair update, revocation request, cross-certification. |